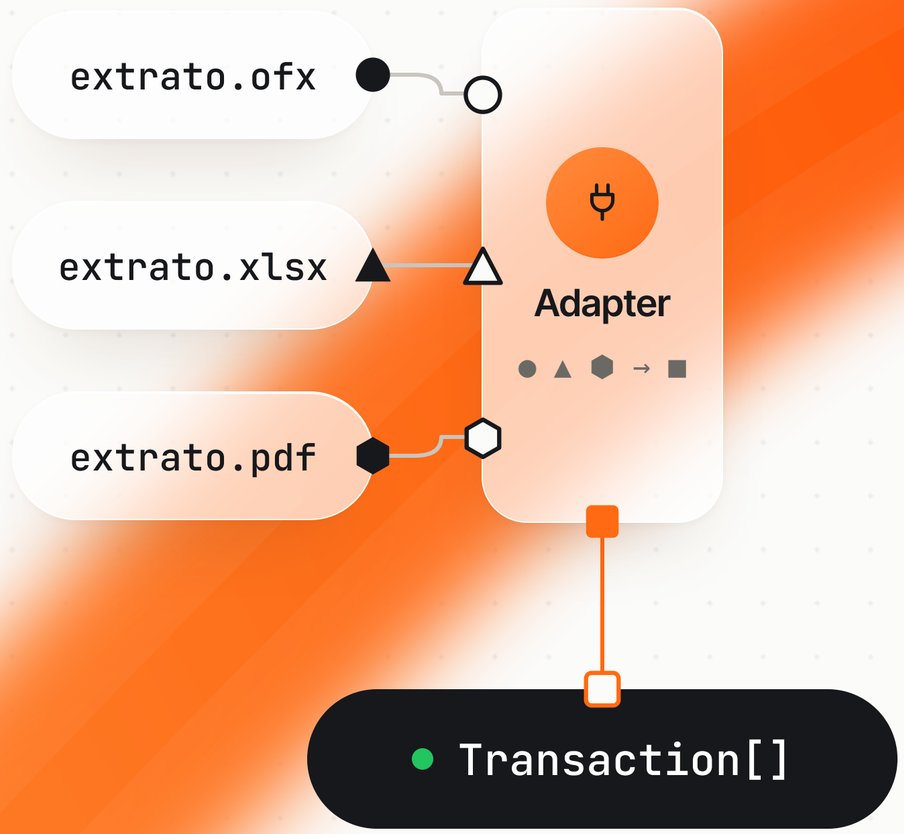
Padrão de projeto estrutural do catálogo GoF

Adapter

Um contrato para importar OFX, Excel e PDF sem mexer nas regras do sistema.

Ver o código →

[Seu nome]



A mesma transação chega de três jeitos

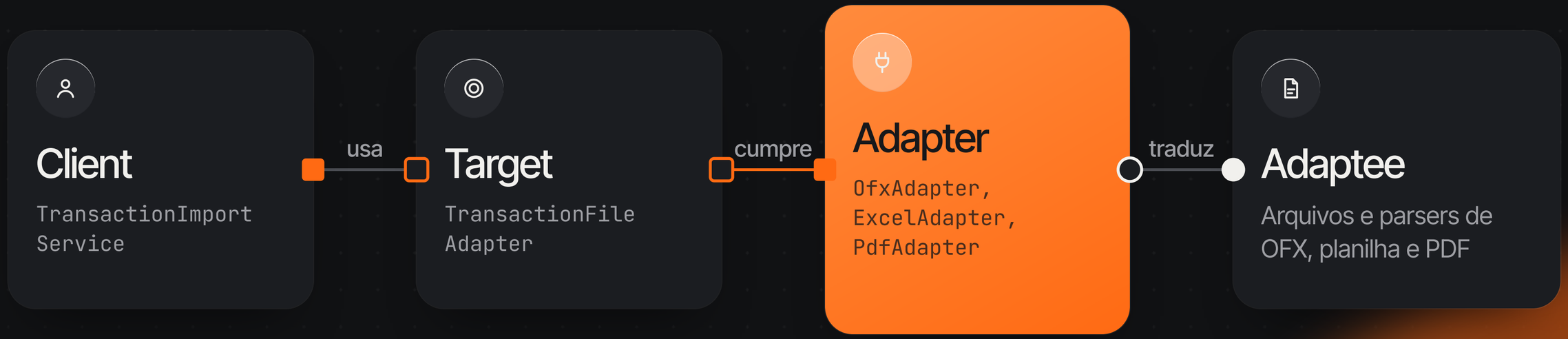
Sem um padrão, o serviço vira um if/else por formato, e cada formato novo mexe nas regras centrais.

● OFX ▲ Excel ⬢ PDF ■ o que o sistema espera



O conceito

O Adapter converte uma interface incompatível na interface que o cliente espera.



Três entradas viram o mesmo objeto



transaction.json

```
{
  date: "2026-10-05",
  description: "Pagamento fornecedor",
  amount: 150, type: "DEBIT",
  externalId: "TX001",
}
```

amount guarda sempre o valor absoluto; **type** diz se é crédito ou débito. É uma convenção do domínio, não do padrão.

Passos 1 e 2 de 5

O modelo e o contrato

Primeiro definimos o que o sistema entende. Depois, a regra que todo adaptador precisa cumprir.

A interface não menciona Excel, PDF nem OFX.

1 · transaction.ts

```ts
type TransactionType = "CREDIT" | "DEBIT";

interface Transaction {
  date: string;
  description: string;
  amount: number;
  type: TransactionType;
  externalId?: string;
}
```

2 · transaction-file-adapter.ts — Target

```ts
interface TransactionFileAdapter {
  parse(content: Buffer): Promise<Transaction[]>;
}
```

Passo 3 de 5: um adaptador por formato

OfxAdapter

ofx.adapter.ts

```ts
class OfxAdapter implements TransactionFileAdapter {
  async parse(content: Buffer): Promise<Transaction[]> {
    const entries = await parseOfxFile(content);
    return entries.map((entry) => ({
      date: normalizeOfxDate(entry.DTPOSTED),
      description: entry.MEMO,
      amount: Math.abs(Number(entry.TRNAMT)),
      type: Number(entry.TRNAMT) < 0 ? "DEBIT" : "CREDIT",
      externalId: entry.FITID,
    }));
  }
}
```

- DTPOSTED → date
- MEMO → description
- TRNAMT → amount
- sinal de TRNAMT → type
- FITID → externalId

Passo 3 de 5: um adaptador por formato

Excel e PDF

excel.adapter.ts

```ts
class ExcelAdapter {
  async parse(content: Buffer) {
    const rows = await parseExcelFile(content);
    return rows.map((row) => ({
      date: normalizeExcelDate(row["Data"]),
      description: row["Histórico"],
      amount: Math.abs(parseMoney(row["Valor"])),
      type: normalizeTransactionType(row["Tipo"]),
    }));
  }
}
```

pdf.adapter.ts

```ts
class PdfAdapter {
  async parse(content: Buffer) {
    const text = await extractPdfText(content);
    const entries = parseStatementLines(text);
    return entries.map((entry) => ({
      date: normalizePdfDate(entry.date),
      description: entry.description,
      amount: Math.abs(parseMoney(entry.amount)),
      type: entry.amount < 0 ? "DEBIT" : "CREDIT",
    }));
  }
}
```

mesma saída: Transaction[]

Passo 4 de 5

Um serviço para todos os formatos

Validação, deduplicação e persistência ficam aqui, fora dos adaptadores.

Recebe a interface, não uma classe concreta.

Por isso aceita qualquer adaptador.

transaction-import-service.ts

```ts
class TransactionImportService {
  async import(
    content: Buffer,
    adapter: TransactionFileAdapter,
  ): Promise<Transaction[]> {
    const transactions = await adapter.parse(content);
    this.validate(transactions);
    // deduplicação, regras de negócio, persistência

    return transactions;
  }
}
```

Passo 5 de 5

Passo 5: escolher o adaptador

import-file.ts

```ts
async function importFile(
  ext: string, content: Buffer,
) {
  const adapter = adapters[ext.toLowerCase()];
  if (!adapter) {
    throw new Error("Formato não suportado");
  }
  const service = new TransactionImportService();
  return service.import(content, adapter);
}
```

O objeto **adapters** já faz uma seleção simples, sem precisar de uma Factory completa.

adapter-resolver.ts

```ts
const adapters: Record<string, TransactionFileAdapter> = {
  ofx: new OfxAdapter(),
  xlsx: new ExcelAdapter(),
  pdf: new PdfAdapter(),
};
```

terminal

```
$ importFile("ofx", ofxBuffer)
✓ OfxAdapter → Transaction[]
$ importFile("xlsx", excelBuffer
✓ ExcelAdapter → Transaction[]
$ importFile("csv", csvBuffer)
✗ Error: Formato não suportado
```



Como a importação flui no projeto

`src/transactions/`

`import/parsers/`

Lê o arquivo, com bibliotecas como SheetJS ou PDF.js.

`ofx · excel · pdf .parser.ts`

`import/adapters/`

Traduz os campos de cada formato para o modelo comum.

`ofx · excel · pdf .adapter.ts`

`domain/`

Regras de negócio, validação, deduplicação e gravação.

`transaction-import-service.ts`

O que o padrão não resolve sozinho

Duplicidades são regra do domínio

Use o FITID do OFX, mas deduplique no serviço, nunca no adaptador.

Valores

Centavos inteiros ou biblioteca decimal.

Datas

Fuso horário e formatos aceitos explícitos.

PDFs

O usuário revisa o que foi extraído.

Erros parciais

Dizer quais linhas falharam e por quê.

Segurança

Validar o conteúdo real e limitar o tamanho.

O Adapter só faz a tradução técnica.

Três padrões que trabalham juntos no projeto

Adapter

Converte OFX, Excel e PDF para um modelo comum.

OfxAdapter, ExcelAdapter, PdfAdapter

Factory

Cria ou seleciona o adaptador certo.

No projeto: o mapa adapters

Strategy

Troca um algoritmo por outro.

No projeto: regras de duplicidade

Veja o código rodando

demo-adapter.mp4



[Vídeo da demonstração]

Roteiro

1. Importar um arquivo OFX
2. A mesma transação em Excel
3. Comparar as duas saídas
4. Adicionar um formato novo

Formato novo? Basta um adaptador novo.

Uma interface, um adaptador por formato e um serviço independente dos arquivos. As regras centrais não mudam.

Perguntas?

Obrigado!

Gamma et al., Design Patterns · Refactoring.Guru · SheetJS · PDF.js · OFX

[Seu nome], [Disciplina], Prof.ª [Nome]

Ex.: o CSV que falhou no terminal

